

SmartReplayOps

Intelligent Trade Replay & Recovery Automation

A production support automation concept demonstrating how banking trade failures caused by MQ and database outages can be detected, classified, and recovered intelligently — with idempotency guarantees, settlement cut-off controls, instrument-aware governance, and immutable audit trails.

LEVEL 1

LEVEL 2

LEVEL 3

LEVEL 4

1. THE PROBLEM

In a banking production environment, trades flow through a distributed pipeline. When any component fails mid-pipeline, trades end up in partial states that require careful, governed recovery to avoid duplicate trades, settlement failures, or reconciliation breaks.

Front Office	→	MQ Queue	→	Trade Processor	→	Database	→	Settlement / Risk / Reporting
--------------	---	----------	---	-----------------	---	----------	---	-------------------------------

Failure Type	Root Cause	Risk if Mishandled
MQ Outage	Queue manager disconnect / network partition	Trade lost in transit — never reaches processor
DB Timeout	Write timeout at BOOK stage	Partial state — commit status uncertain
Partial Publish	Outbound publish fails after booking succeeds	Internal and external state diverge silently
Poison Message	Malformed data / missing reference data	Infinite retry loop consuming processing capacity
Post-cut-off Failure	Failure occurs after settlement window closes	Booking succeeds but settlement silently fails downstream

Without automation: a support analyst manually investigates each failure, queries the database, checks MQ logs, determines replay scope, and executes the replay — a process taking 30–90 minutes per trade with significant risk of introducing duplicate trades or missing downstream reconciliation gaps.

2. FOUR RECOVERY LEVELS

Level 1 MQ Auto-Recovery

Detects queue manager disconnect via heartbeat failure and connection timeout. Automatically reconnects consumers and restarts listeners. Dashboard shows live transition: DISCONNECTED → RECOVERING → CONNECTED.

Consumer-side deduplication fires on reconnect. Before re-processing any message, the engine checks if the trade_ref already exists with status COMPLETED or higher. If found, the ACK is sent and processing is skipped — preventing duplicate processing caused by lost ACK acknowledgements.

Level 2 Safe Replay Verification

Before replaying any failed trade, the engine verifies: no database commit present, no outbound message published, and rollback was clean. Only then does it approve replay — preventing the most dangerous outcome, a duplicate trade.

Idempotency is enforced via a UNIQUE(trade_ref, workflow_stage) database constraint, not by querying the downstream_state table alone. The replay uses upsert semantics. A constraint violation means a duplicate has been detected and is rejected at the database level before any processing begins.

Level 3 Partial-State Detection

Checks downstream_state to determine replay scope. If a trade is booked internally but the outbound publish failed, replay scope is set to PUBLISH_ONLY — not a full pipeline replay. This prevents re-booking a trade that was already committed.

Before executing a PUBLISH_ONLY replay, the engine performs a downstream health check: it verifies the settlement system inbound queue has active consumers. If consumer count is zero, replay is deferred (REPLAY_DEFERRED) — not fired into a silent queue that would produce a false SUCCESS status.

Level 4 Checkpoint-Based Resumption

Every trade stores its last successful workflow_stage as a checkpoint. On replay, processing resumes from that exact stage — not from the beginning. A trade that failed at PUBLISH resumes from PUBLISH, not from VALIDATE.

A pre-replay amendment check is performed: if a newer version of the same trade_ref exists with a later timestamp (e.g. a price amendment received after the original failure), the original trade is marked SUPERSEDED and replay is blocked — preventing the old price from overwriting the amendment.

3. GOVERNANCE & OPERATIONAL CONTROLS

Replay decisions are governed by a two-dimensional framework: an instrument-type approval matrix that reflects the real risk profile of each asset class, and settlement cut-off windows that determine whether auto-replay is permissible at the time of the failure.

Instrument	Auto-Replay Limit	Above Limit	Rationale
EQUITY	≤ \$500,000	Manual approval	T+1 DTCC settlement — bilateral risk is low
FX	≤ \$2,000,000	Manual approval	Bilateral reversible settlement via CLS T+2
RATES	Always manual	Always manual	Notional ≠ exposure. A \$900K swap can carry \$50M+ DV01 risk
CREDIT	Always manual	Always manual	CDS sensitivity — counterparty risk varies independently of notional

Why instrument type matters: a single \$1M notional threshold treats an equity trade and an interest rate swap identically. A RATES swap at \$900K notional can carry \$50M+ of interest rate exposure (DV01). Governance must reflect actual risk, not face value.

Settlement cut-off windows are enforced before any auto-replay is permitted. A trade replayed after its cut-off will succeed at the booking level but the settlement instruction will miss the downstream batch — producing a false REPLAY_SUCCESS with no actual settlement.

Asset Class	Settlement Route	Cut-off Time	Action if Past Cut-off
EQUITY	T+1 via DTCC	16:30 ET	REQUIRES_AMENDMENT — front office notified
FX	T+2 via CLS	17:00 ET	REQUIRES_AMENDMENT — front office notified
RATES	T+2 via LCH	15:00 ET	REQUIRES_AMENDMENT — front office notified
CREDIT	T+3 via DTCC	14:00 ET	REQUIRES_AMENDMENT — front office notified

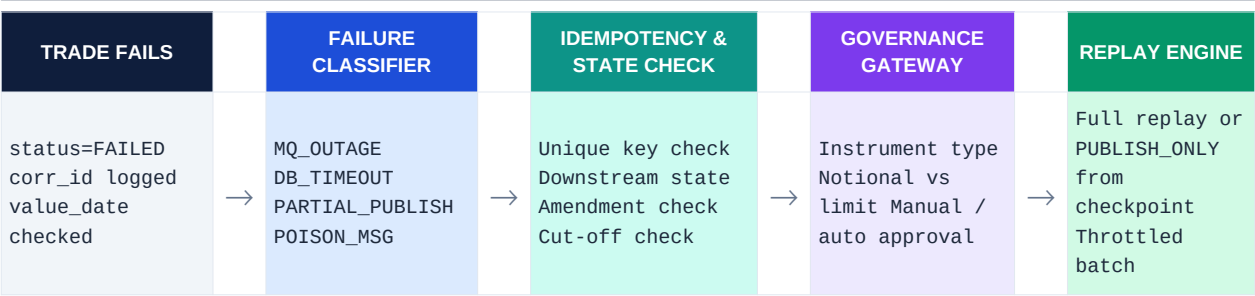
4. POISON MESSAGE HANDLING & QUARANTINE

A poison message is one that fails processing on every attempt regardless of infrastructure state — caused by malformed data, a missing BIC code, or a violated business rule. Without a maximum retry limit, the replay engine will loop indefinitely, consuming capacity and masking the root cause.

Retry Count	Engine Action	Status
1	Auto-retry with 5-second backoff	FAILED
2	Auto-retry with 15-second backoff	FAILED
3 (MAX)	Reclassify failure type as POISON_MESSAGE. Route message to DLQ. Raise manual investigation flag.	QUARANTINED

The DLQ (dead letter queue) is pre-declared in the broker topology. Quarantined messages are routed there with their full metadata preserved. A support analyst can inspect, correct the reference data (e.g. update the SSI/BIC), and replay from the DLQ once the root cause is resolved.

5. RECOVERY DECISION FLOW



6. SAMPLE AUDIT TRAIL

Every state transition writes an immutable row to the audit log. The correlation ID links the original message through its entire recovery chain. The log captures not only replay actions but also idempotency checks, governance decisions, cut-off assessments, and decisions not to replay — all with timestamp and initiator.

Full recovery — TRD-012 (EQUITY, BUY 600 QQQ, MQ Outage):

Time	Event	Initiator	Detail	Outcome
10:42:01	TRADE_RECEIVED	PRODUCER	BUY 600 QQQ @ 438.90 — idempotency key registered	SUCCESS
10:42:03	TRADE_FAILED	PROCESSOR	MQ connection lost during PUBLISH stage	FAILED
10:44:15	IDEMPOTENCY_CHECK	AUTO_ENGINE	UNIQUE(TRD-012:BOOKED) — no prior commit confirmed via DB constraint	SUCCESS
10:44:15	GOVERNANCE_CHECK	AUTO_ENGINE	EQUITY \$263,340 ≤ \$500,000 threshold — auto-replay approved	SUCCESS
10:44:15	REPLAY_INITIATED	AUTO_ENGINE	Scope: FULL_FROM_CHECKPOINT Resume: ENRICHED Batch slot: 1/10	PENDING
10:44:17	STAGE_BOOKED	PROCESSOR	DB upsert — idempotency key accepted, no prior commit present	SUCCESS
10:44:18	STAGE_PUBLISHED	PROCESSOR	Consumer count: 3 active — outbound publish successful	SUCCESS
10:44:19	REPLAY_SUCCESS	AUTO_ENGINE	Trade fully recovered via FULL_FROM_CHECKPOINT	SUCCESS

Cut-off block — TRD-014 (EQUITY, BUY 100 TSLA, failure after 16:30 cut-off):

Time	Event	Initiator	Detail	Outcome
16:32:00	TRADE_RECEIVED	PRODUCER	BUY 100 TSLA @ 248.75 — value date today	SUCCESS
16:32:02	TRADE_FAILED	PROCESSOR	DB write timeout at BOOK stage	FAILED

16:32:02	CUTOFF_CHECK	AUTO_ENGINE	EQUITY cut-off 16:30 ET — current time 16:32 — PAST CUT-OFF	FAILED
16:32:02	REPLAY_BLOCKED	AUTO_ENGINE	Auto-replay blocked — settlement would miss DTCC batch for T+1	ACTIVE
16:32:03	FRONT_OFFICE_ALERT	AUTO_ENGINE	Status set to REQUIRES_AMENDMENT — front office notification sent	ACTIVE

7. ADDITIONAL FAILURE SCENARIOS

The following scenarios extend the core four levels and must be considered in any production-grade recovery system. Each represents a real failure mode observed in banking distributed systems.

Scenario	What Happens	Correct Handling
Network partition (ACK lost)	MQ delivers message. Processor starts. Network drops before ACK is sent. MQ re-queues. Processor receives it again.	Consumer-side deduplication: check if trade_ref is already COMPLETED before processing. If yes, send ACK and skip — no re-processing.
Amendment after failure	TRD-007 fails at 10:45. At 11:10 a price amendment arrives and is processed. At 11:30 replay engine picks up TRD-007 at the old price.	Pre-replay check: does a later version of trade_ref exist? If yes, mark original SUPERSEDED and block replay. Log the supersession reason.
Downstream system degraded	PUBLISH_ONLY replay fires. Settlement system has no active consumers. MQ accepts the message. Engine reports REPLAY_SUCCESS. Nothing settles.	Consumer count check before publish-only replay. Zero consumers → status set to REPLAY_DEFERRED. Retry when consumers are active.
SSL/TLS cert expiry	MQ disconnects. Auto-reconnect loop fires. Every attempt fails with SSL handshake error. The reconnect loop spins indefinitely.	Classify reconnect failure reason. Repeated SSL errors → flag CERT_EXPIRY, alert infrastructure team, stop reconnect loop.
Trade dependency violation	Trade B amends Trade A. Both fail. Replay in wrong order: Trade B replayed before Trade A exists in the system. Booking system rejects it.	depends_on_trade_id field on trades table. Replay engine sequences by dependency chain first, then by original submission timestamp.
Replay under high load	300 trades fail during a 20-minute MQ outage. On recovery, all 300 are replayed simultaneously, flooding the booking and risk systems.	REPLAY_BATCH_SIZE config (e.g. 10 per second). Prioritise by value date proximity to cut-off, then by notional. Monitor downstream health metrics during replay.

“Senior production support is not about closing tickets faster — it is about designing systems that detect, classify, and recover intelligently, with idempotency guarantees, settlement awareness, and the governance controls that regulators and change management processes require.”